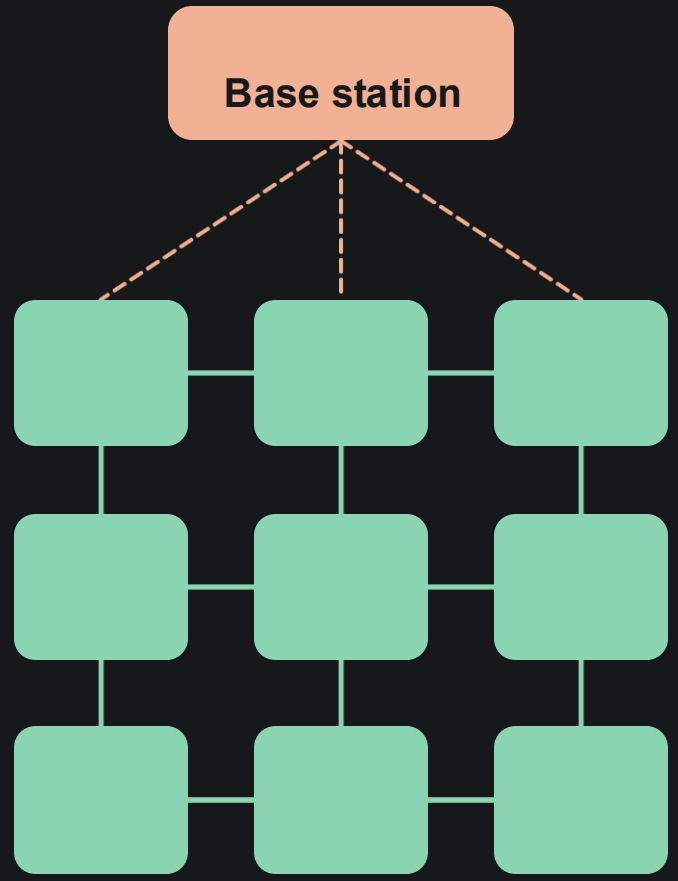
FIT3143 LAB 3 · WEEK 12

Distributed Wireless Sensor Network of EV Charging Nodes

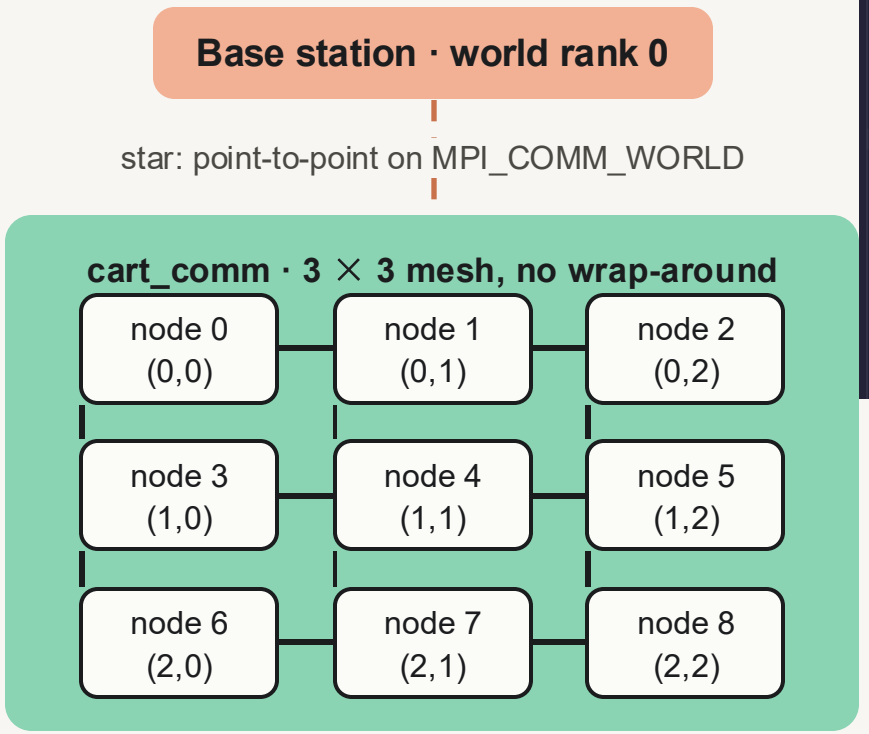
Open MPI + OpenMP · a Cartesian mesh of charging nodes in a star with one base station

Charmaine Chee Hing Yi · 33520399 · cche0258@student.monash.edu
Nazara Glaukofani Lopulisa · 35135271 · nnaz0017@student.monash.edu



TASK 1 · ARCHITECTURE

Star + mesh: the base station sits outside the grid




```c
/* base station is NOT part of the grid: split it out (star vs mesh) */
MPI_Comm nodes_comm;
MPI_Comm_split(MPI_COMM_WORLD, world_rank == BASE_RANK ? MPI_UNDEFINED : 0,
               world_rank, &nodes_comm);

MPI_Datatype report_t = build_report_type();
MPI_Datatype reply_t  = build_reply_type();

if (world_rank == BASE_RANK) {
    clock_sync_base(cfg.m * cfg.n);
    base_run(&cfg, report_t, reply_t);          /* nodes_comm == MPI_COMM_NULL */
} else {
    int dims[2] = { cfg.m, cfg.n }, periods[2] = { 0, 0 };   /* mesh, not torus */
    MPI_Comm cart_comm;
    MPI_Cart_create(nodes_comm, 2, dims, periods, 0 /* no reorder */, &cart_comm);
    double offset = clock_sync_node();
    node_run(&cfg, cart_comm, report_t, reply_t, offset);
    MPI_Comm_free(&cart_comm);                  /* collective over nodes only */
    MPI_Comm_free(&nodes_comm);
}
```

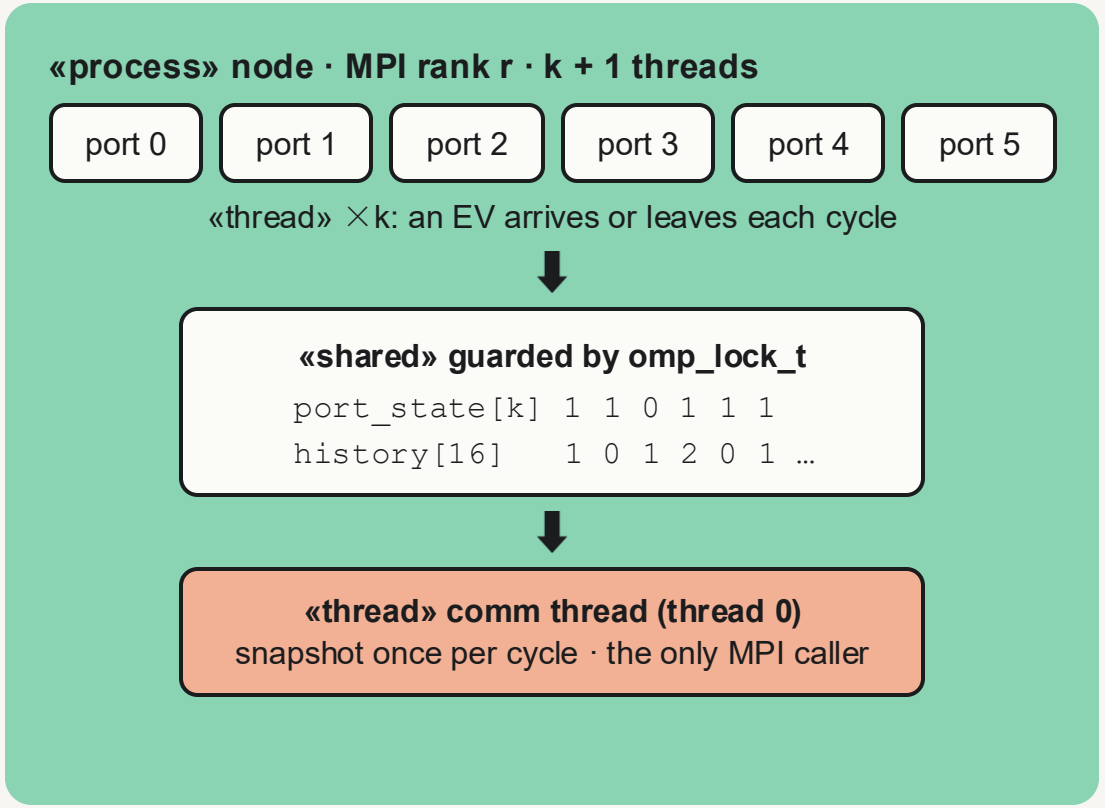
```c
static void comm_thread(Node *nd) {
    const Config *cfg = nd->cfg;
    double poll = cfg->poll_ms / 1e3;
    MPI_Comm_rank(nd->cart, &nd->cart_rank);
    MPI_Cart_coords(nd->cart, nd->cart_rank, 2, nd->coords);
    MPI_Cart_shift(nd->cart, 0, 1, &nd->nbr[UP],   &nd->nbr[DOWN]);
    MPI_Cart_shift(nd->cart, 1, 1, &nd->nbr[LEFT], &nd->nbr[RIGHT]);
    nd->num_nbrs = 0;
    for (int d = 0; d < NUM_DIRS; d++) {
        nd->nbr_coords[d][0] = nd->nbr_coords[d][1] = -1;
        if (nd->nbr[d] != MPI_PROC_NULL) { MPI_Cart_coords(nd->car
        nd->req_req[d] = nd->resp_req[d] = MPI_REQUEST_NULL;
    }
```

Tag	Direction	MPI calls
NBR_REQ / RESP	node ↔ node	Isend · Iprobe → Recv
REPORT	node → base	Isend · pre-posted Irecv
REPLY	base → node	Isend · Iprobe → Recv
TERM / DONE	base ↔ node	Isend ×N · Send

Changed from our week 6 design: the base station is no longer a grid member, so the Cartesian topology holds charging nodes only. · 02

TASK 1 · INSIDE ONE CHARGING NODE

One MPI process, k port threads, one shared array




```c
omp_set_dynamic(0);
#pragma omp parallel num_threads(cfg->k + 1)
{
    int tid = omp_get_thread_num();
    if (tid == 0) comm_thread(&nd);          /* master = FUNNELED */
    else          port_thread(cfg, tid - 1, nd.cart_rank);
}   /* implicit barrier: every port thread has returned */
```

Nazara Glaukofan

```c
/* ---------- charging port thread ---------- */
static void port_thread(const Config *cfg, int port, int cart_rank) {
    unsigned int seed = (unsigned int)(time(NULL) ^ (cart_rank * 7919) ^ (port * 104729));
    /* stagger ports so they do not all change in lock-step */
    sleep_seconds(cfg->cycle * port / (double)cfg->k);
    while (!should_stop()) {
        /* EV arrives / departs: port is in use with probability busy_pct */
        int in_use = (rand_r(&seed) % 100) < cfg->busy_pct;
        omp_set_lock(&state_lock);
        port_state[port] = in_use;
        omp_unset_lock(&state_lock);

        /* sleep one cycle in small slices so termination is noticed quickly */
        for (double slept = 0; slept < cfg->cycle && !should_stop(); slept += 0.05)
            sleep_seconds(0.05);
    }
}
```

Nazara Glaukofani Lopulisa, yesterday • Modified and added second version of wsn to acc…

Congested = free ports < threshold. Mutual exclusion stops a race condition, so the snapshot is never half-updated.

Every node also writes its own log file (node_r.log) with one row per cycle in the Table 1 format. · 03

TASK 1 · WHEN A NODE IS CONGESTED

Ask the neighbours first, alert the base only if all are full

Node 4 asks its 4 neighbours

0	**1** free 0	2
3 free 2 · room	**4** free 1	**5** free 0
6	**7** free 1	8

Round ends when all reply, or after 500 ms (no reply = possibly dead)

```
483
484     char action[128];
485     int best = best_neighbour(nd, rd->avail);
486     if (best >= 0) {
487         /* a neighbour has room: redirect locally, no base station traffic */
488         nd->st.local_redirects++;
489         snprintf(action, sizeof action, "CONGESTED -> redirect EVs to node %d (%d,%d) avail=%d",
490                  nd->nbr[best], nd->nbr_coords[best][0], nd->nbr_coords[best][1], rd->avail[best]);
491     } else if (terminating) {
492         snprintf(action, sizeof action, "CONGESTED, neighbours full (shutting down: no alert)");
493     } else {
494         /* node and all (responding) neighbours congested: alert the base station */
495         send_report(nd, rd->cycle, &rd->tmv, rd->my_avail, rd->states, rd->avail, rd->msgs, rd->bytes, xchg);
496         snprintf(action, sizeof action, "CONGESTED, all neighbours full -> ALERT base station");
497     }       Nazara Glaukofani Lopulisa, yesterday • Modified and added second version of wsn to acc…
498     print_row(nd, &rd->tmv, rd->my_avail, rd->states, rd->avail, xchg * 1e3, action);
499 }
```

```
Nazara Glaukofani Lopulisa, yesterday | author (Nazara Glaukofani Lo
typedef struct {
    int cart_rank, world_rank;
    int coords[2];
    int cycle;
    int year, month, day, hour, minute, second;
    int k, availability;
    int port_state[MAX_PORTS];
    int history[HISTORY_LEN];
    int num_neighbours;
    int nbr_rank[NUM_DIRS];
    int nbr_coords[NUM_DIRS][2];
    int nbr_avail[NUM_DIRS];
    int nbr_no_response;
    int msgs_with_neighbours;
    int bytes_with_neighbours;
    /* doubles */
    double send_wtime;
    double nbr_comm_time;
    double last_base_rtt;
} Report;       Nazara Glaukofani Lopulisa, yeste
```

The Report: one message, 480 B, MPI_Type_create_struct

- detection timestamp (date + time) and cycle number
- node rank, world rank and (x, y) coordinates
- own free ports, every port's state, last 16 availability values
- each neighbour's rank, coordinates and free ports (or "no response")
- neighbour messages and bytes exchanged, neighbour round time
- send time (for one-way delay), previous alert round trip

node_4.log: 13:41:11 | free 1 | nbrs 0 1 2 0 | 2.534 ms | redirect EVs to node 3 avail=2 · 04

TASK 1 · EFFICIENCY AND TERMINATION

Efficient while running, deadlock-free when stopping

On-demand, not periodic
216 of 412 congested cycles solved locally

Non-blocking sends (MPI_Isend)
comm thread never blocks on a slow peer

One message per report
480 B derived datatype, not many sends

Check, then sleep 1 ms
no busy-waiting on Iprobe / Test

1 Base: time up → cancel its posted Irecv
2 Base: Isend TERM to every node
3 Node: finish its open round; port threads exit
4 Node: MPI_Ibarrier, still answering neighbours
5 Node: Send DONE + statistics; base counts all N

```
if (on_demand) {
    /* non-blocking barrier: keep answering neighbours until EVERY node has finished its
     * last round, so nobody is left waiting for an answer that never comes */
    MPI_Request bar;
    int done = 0;
    MPI_Ibarrier(nd->cart, &bar);
    while (!done) {
        service_neighbours(nd, NULL, !nd->dead);
        MPI_Test(&bar, &done, MPI_STATUS_IGNORE);
        if (!done) sleep_seconds(poll);
    }
    for (double end = MPI_Wtime() + 0.02; MPI_Wtime() < end; sleep_seconds(poll))
        service_neighbours(nd, NULL, 0);              /* drain anything still in flight */
    MPI_Waitall(NUM_DIRS, nd->req_req, MPI_STATUSES_IGNORE);
    MPI_Waitall(NUM_DIRS, nd->resp_req, MPI_STATUSES_IGNORE);
}
```

208 CAAS runs: all clean shutdowns, sent = received

Then MPI_Comm_free, MPI_Type_free, free() on every buffer, MPI_Finalize. No Ctrl+C or kill. · 05

TASK 2 · BASE STATION MODEL

The base station: one process, one thread

1 Post
Irecv from any node for REPORT

→

2 Listen
MPI_Test every 1 ms, sleep if idle

→

3 Update
knowledge tables

→

4 Decide
find_nearest()

→

5 Reply
Isend REPLY, then re-post Irecv → 2

→

6 Log
alert to file, after the reply has left

Knowledge tables (one entry per node)

last_report[j] · when node j last alerted

last_full[j] · last known full (it alerted, or a neighbour said so)

last_silent[j] · a neighbour said j did not answer

Single thread: no thread creation or destruction, and no MPI thread-safety overhead.

base_station.log summary · real 3 × 3 run, 60 s

```
alerts    : 192 answered, 1 late
replies   : 6 nearest nodes, 186 none nearby
nbr msgs  : sent=2190 recv=2190
base msgs: 403 (report 193, reply 192,
           term 9, done 9)
alert round trip  avg 2.246 ms
base processing   avg 0.001 ms
```

Each alert in the log also records detection time, port states, history, every neighbour's availability and last report, message counts and the decision. · 06

TASK 2 · BASE STATION MODEL

The base station: one process, one thread

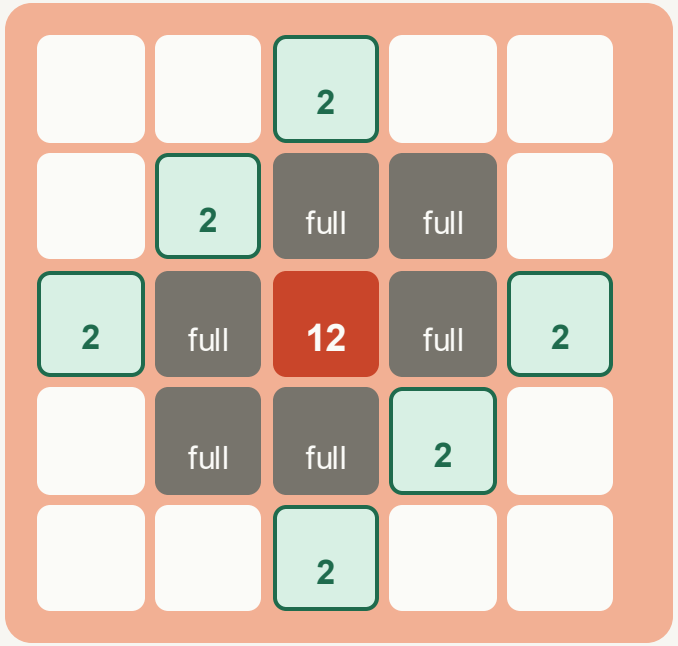
The listening loop, `task2_base.c` lines 272–300

```c
if (cfg->base_variant == BV_IRECV_TEST) {
    MPI_Irecv(&rep, 1, report_t, MPI_ANY_SOURCE, TAG_REPORT, MPI_COMM_WORLD, &req);
} else if (cfg->base_variant == BV_PERSISTENT_TESTSOME) {
    prep = malloc(N * sizeof(Report));
    preq = malloc(N * sizeof(MPI_Request));
    idx  = malloc(N * sizeof(int));
    if (!prep || !preq || !idx) {
        fprintf(stderr, "base station: out of memory\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    for (int i = 0; i < N; i++)
        MPI_Recv_init(&prep[i], 1, report_t, i + 1, TAG_REPORT, MPI_COMM_WORLD, &preq[i]);
    MPI_Startall(N, preq);
}

/* ---------- periodically listen for reports until the simulation ends ---------- */
while (MPI_Wtime() - b.start < cfg->duration) {
    int got = 0;
    if (cfg->base_variant == BV_IPROBE_RECV) {
        /* Iprobe: is a report waiting? Only then call Recv */
        MPI_Iprobe(MPI_ANY_SOURCE, TAG_REPORT, MPI_COMM_WORLD, &got, &st);
        if (got) {
            MPI_Recv(&rep, 1, report_t, st.MPI_SOURCE, TAG_REPORT, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            handle_report(&b, &rep, MPI_Wtime());
        }
    } else if (cfg->base_variant == BV_IRECV_TEST) {
        MPI_Test(&req, &got, MPI_STATUS_IGNORE);
        if (got) {
            handle_report(&b, &rep, MPI_Wtime());
            MPI_Irecv(&rep, 1, report_t, MPI_ANY_SOURCE, TAG_REPORT, MPI_COMM_WORLD, &req);
        }
```

TASK 2 · NEAREST AVAILABLE NODES

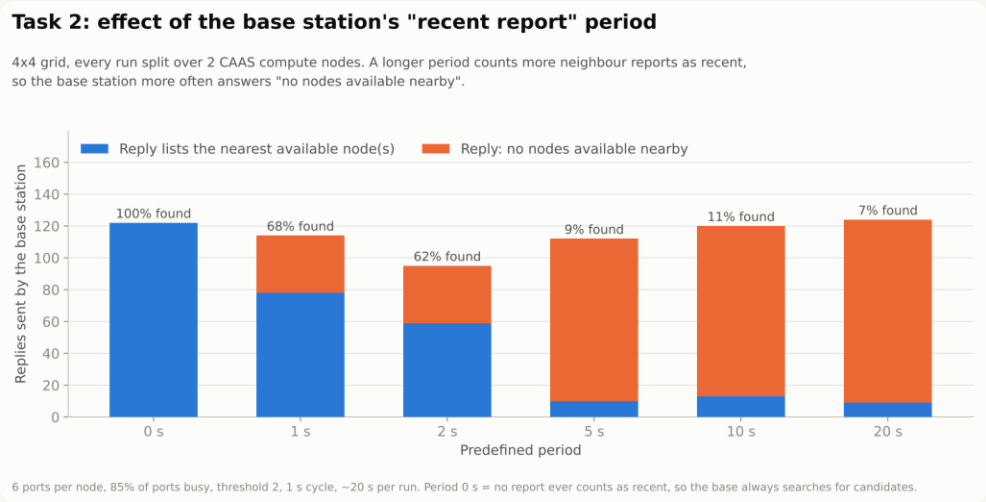
Search outward ring by ring, reply with every nearest node



Alert from node 12 (5 × 5). Grey = known full within the period. Reply: all 6 nodes at distance 2.

Every neighbour alerted within the period → reply “no nodes available nearby”

Otherwise → every node at the smallest Manhattan distance not known full or silent



0 wrong

72,036 base station decisions replayed from the logs (verify_replies.py). 350 sit on the 1 ms period boundary.

We use period = 5 s (5 cycles). At 85% load most areas really are full, so “none nearby” is usually the correct answer. · 07

TASK 2 · NEAREST AVAILABLE NODES

Search outward ring by ring, reply with every nearest node

`find_nearest`, `task2_base.c` lines 58–88

```c
if (recent == nbrs) {
    r->status = REPLY_NONE_NEARBY;
    return recent;
}

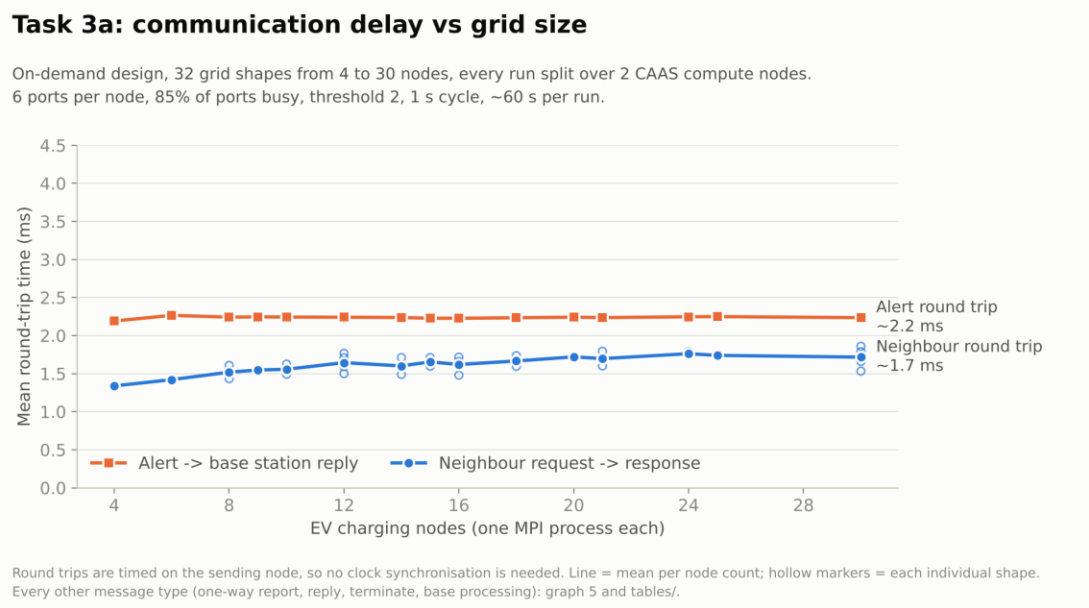
/* nearest = smallest Manhattan distance (hops in the mesh) among nodes not known full/silent */
int best = INT_MAX;
for (int j = 0; j < b->N; j++) {
    int dist = abs(j / n - r0) + abs(j % n - c0);
    if (dist > 0 && dist < best && now - b->last_full[j] > cfg->period
        && now - b->last_silent[j] > cfg->period) best = dist;
}
if (best == INT_MAX) {
    r->status = REPLY_NONE_IN_GRID;
    return recent;
}

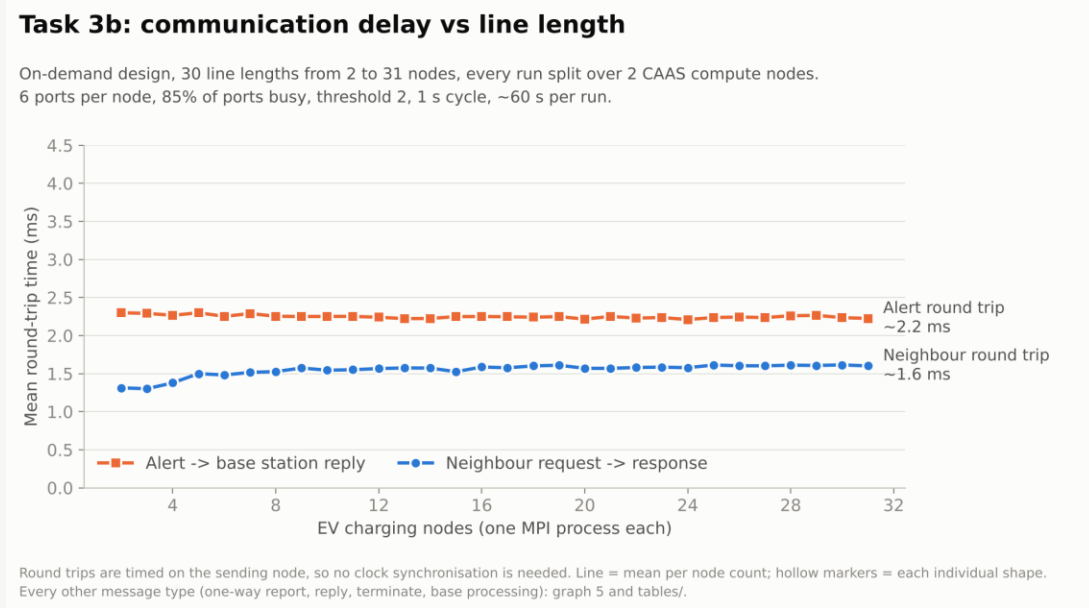
r->status = REPLY_FOUND;
r->distance = best;
for (int j = 0; j < b->N; j++) {   /* report ALL nodes tied at the nearest distance */
    int dist = abs(j / n - r0) + abs(j % n - c0);
    if (dist != best || now - b->last_full[j] <= cfg->period || now - b->last_silent[j] <= cfg->period)
        continue;
    if (r->count < MAX_CAND) {
        r->rank[r->count] = j;
        r->coords[r->count][0] = j / n;
        r->coords[r->count][1] = j % n;
        r->count++;
    }
    r->found++;
}
```

TASK 3 · GRID AND LINE RESULTS

Delay stays flat as the network grows

62 CAAS runs, each split over 2 machines · 3a: 32 grid shapes (2×2, 2×3, 3×2 … 5×6, 6×5, 3×10, 2×15) · 3b: 30 lines (1×2 … 1×31) · 60 s each, 26,526 alerts timed





a) Grid, 4 → 30 nodes

Alert round trip 2.2–2.3 ms at every size. Neighbour round trip 1.3 → 1.8 ms, levelling off once most nodes have 4 neighbours.

b) Line, 2 → 31 nodes

Same flat alert round trip. Neighbour 1.3 → 1.6 ms: at most 2 neighbours, so slightly lower than the grid.

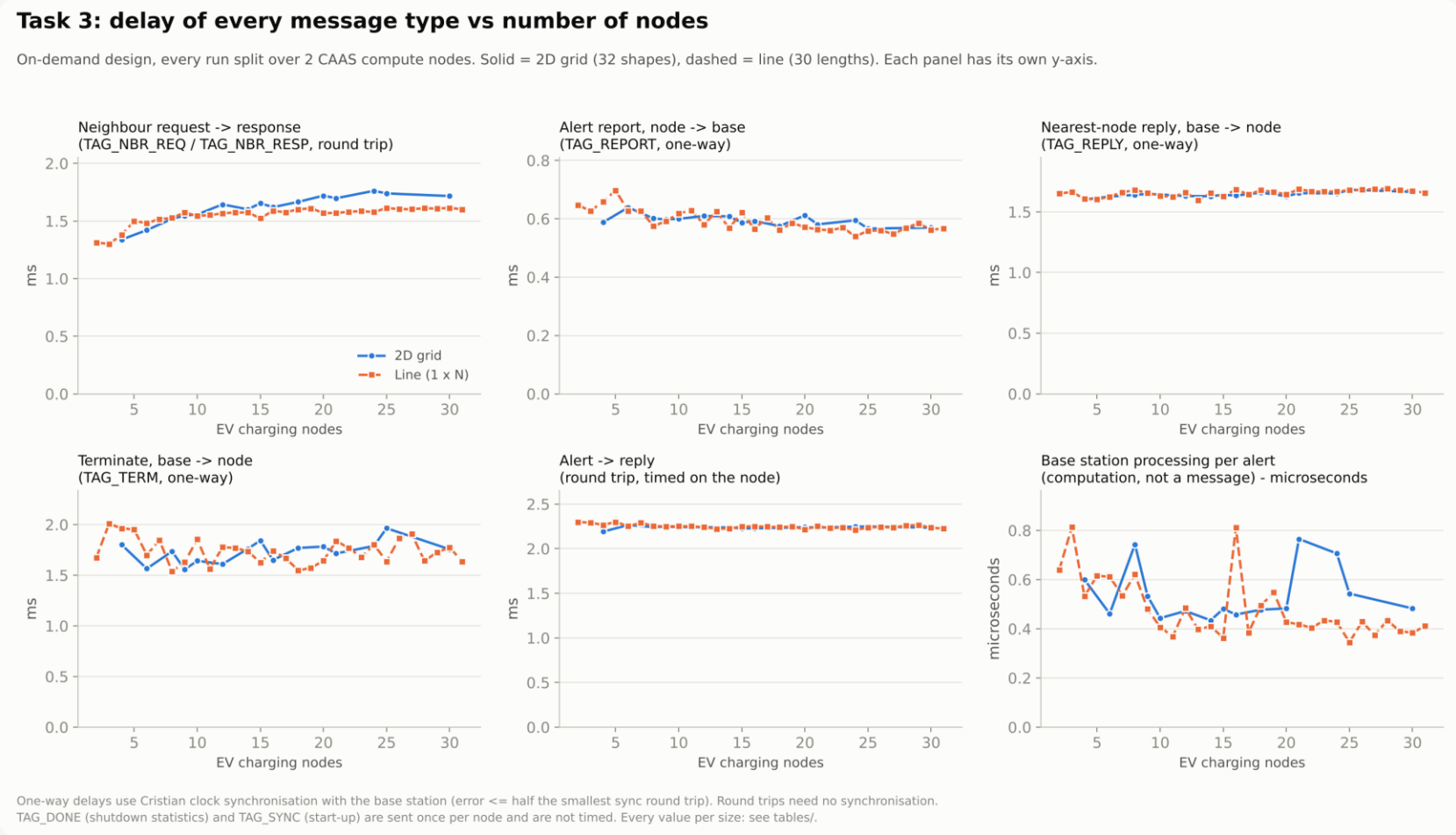
Why: weak scaling

Work per node is constant; every message is point-to-point (≤ 4 neighbours, 1 hop to the base). The floor is our 1 ms polling.

Scaling limit: alerts grow with N but there is one base station, so at large N it would queue (central bottleneck). · 08

TASK 3 · EVERY MESSAGE TYPE

Each message type, on grids and lines



Report → base: 0.6 ms
lands in the base station's posted Irecv

Reply, terminate: ≈ 1.7 ms
the node checks, then sleeps 1 ms: polling latency dominates

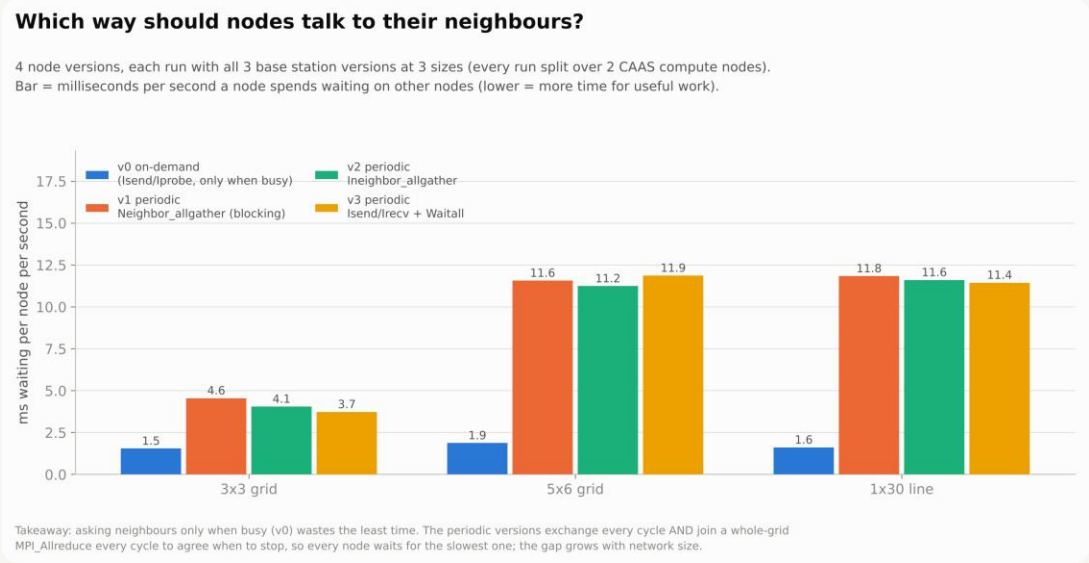
Base processing: < 1 µs
computation is negligible; the delay is communication

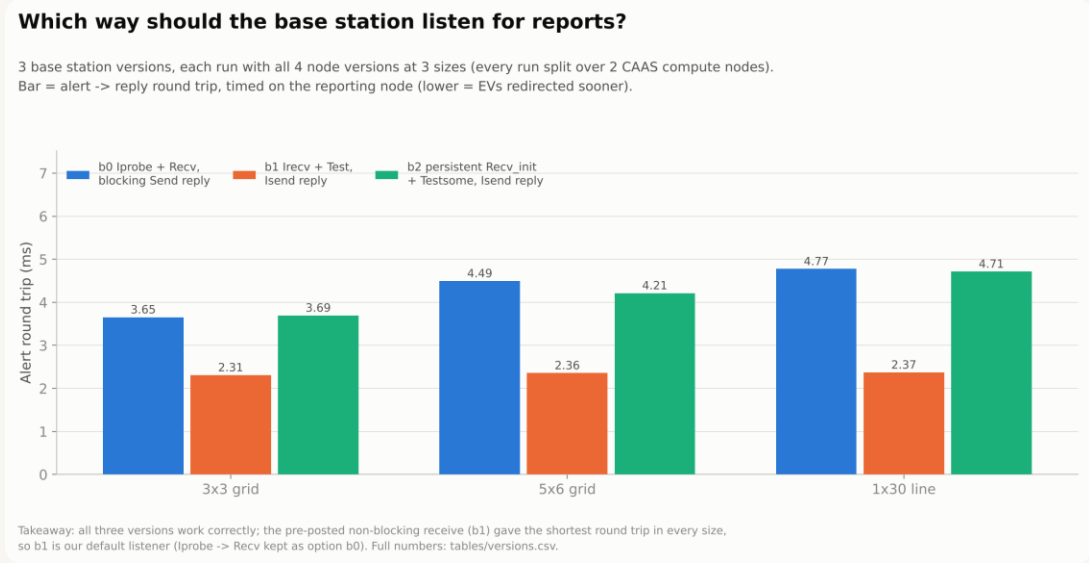
No delay grows with N beyond the neighbour-count effect
up to 31 processes on 2 machines

One-way delays use Cristian's clock synchronisation at start-up; round trips are timed on one clock. Full numbers per size: graphs/tables/. · 09

TASK 3 · DESIGN VERSIONS

We measured alternatives before choosing





4 node versions, blocking and non-blocking. On-demand waits 1.5–1.9 ms per node per second; periodic 3.7–11.9 ms, because a whole-grid MPI_Allreduce each cycle makes every node wait for the slowest (load imbalance).

3 base listeners. Irecv + Test is fastest at every size (≈ 2.3 vs 3.6–4.8 ms), so it is our default. The whole gap is the report leg (0.6 vs 1.7 ms); the reply leg is the same.

3 sizes (3×3, 5×6, 1×30) × 4 node versions × 3 base versions = 36 runs, 20 s each, on 2 CAAS machines. · 10

CONCLUSION

Limitations and future work

Limitations

- One run per configuration: no error bars yet
- Delay floor is our 1 ms polling, not the network
- One base station: a central bottleneck and single point of failure
- CAAS cap of 32 tasks: at most 31 nodes tested
- Simulated ports: random arrivals at a fixed busy %
- Dead nodes found only by a 500 ms neighbour timeout

Future work

- Pre-post the node's receive for base replies, and try 0.1 ms polling
- Find out why persistent Testsome gains less than Irecv + Test
- 3–5 repeats per size with error bars; larger grids
- Heartbeats to detect dead nodes; a backup base station

Thank you. Appendix A maps every requirement to its function and lines in wsn_ver3.c. · 11

APPENDIX A

Where each requirement is in the code

Requirement	File: function	Lines
Base outside the grid, Cartesian topology	wsn_main.c: main (MPI_Comm_split, MPI_Cart_create)	73–87
Neighbours with Cart_shift	task1_node.c: comm_thread	337–338
Port threads + shared array, FUNNELED	task1_node.c: node_run, port_thread, snapshot_availability	507–512, 33–63
Ask neighbours when congested	task1_node.c: service_neighbours, start_round, finish_round	175–267
Report = one message, derived datatype	wsn_common.c: build_report_type · task1_node.c: send_report	30–40 · 124–163
Base: listen periodically	task2_base.c: base_run (Irecv + Test loop)	271–315
Base: nearest nodes, predefined period	task2_base.c: find_nearest, handle_report	46–90, 141–186
Base: logging key metrics	task2_base.c: log_report, base_run summary	92–139, 384–416
Termination + graceful shutdown	task2_base.c: base_run · task1_node.c: comm_thread	318–380 · 449–476
Clock sync for one-way delays	wsn_common.c: clock_sync_node, clock_sync_base	73–96
Reply correctness check	task3/verify_replies.py	whole file

AI Declaration

whats the difference between Iprobe+Recv and Irecv+Test again?

Locate the two listener versions in the code ›

Both let the base station check for reports without ever getting stuck waiting. The difference is the order: **look first, then receive** versus **set up the receive first, then check whether it's filled**.

`Iprobe` + `Recv`**: look first, then receive** (version b0, lines 1029–1032)

1. `MPI_Iprobe` asks MPI: "has a report arrived?" It answers yes or no straight away and doesn't take the message.
2. Only if the answer is yes, `MPI_Recv` actually receives it into the `rep` buffer.

Until the base station asks, an arrived message waits inside MPI's internal storage, and `Recv` then copies it into your buffer. So each report costs two calls: one to look, one to take.

`Irecv` + `Test`**: receive set up in advance** (version b1)

1. Before any report arrives, `MPI_Irecv` (line 1010) tells MPI: "when a report comes, put it straight into `rep`." It returns immediately; nothing has been received yet.
2. Each time round the loop, `MPI_Test` (line 1035) asks: "has that receive been filled yet?"
3. Once a report is in, it's handled, and a new `Irecv` is posted for the next one (line 1038).

Because the receive is already waiting, MPI can pu ↓ incoming message directly into your buffer, and each check is a single call.

We used Claude (Anthropic) as an assistant throughout this assignment. We used it to help design and write parts of the C code, including reworking our initial Task 1 code into the final on-demand neighbour protocol, completing the Task 2 base station logic, adding the alternative node and base station versions. We also used it to adapt our Slurm job for running the Task 3 experiments on CAAS, to write the Python script that produces the graphs and tables and to help plan the presentation slides. All experiments were run by us on CAAS, and the results come from those runs. We reviewed, tested and questioned the generated code and explanations, asked for clarification of the MPI and OpenMP concepts involved, and made the final decisions on the design, such as switching the base station to a pre-posted `MPI_Irecv` with `MPI_Test` after measuring it.

what does report, reply, term, done mean?

They're the names of the four kinds of messages that travel between a charging node and the base station. In MPI, every message carries a small number called a **tag**, and the receiver uses it to tell what kind of message has arrived. Your code gives each kind its own tag: `TAG_REPORT = 1`, `TAG_REPLY = 2`, `TAG_TERM = 3` and `TAG_DONE = 4` in `wsn_ver3.c`. That's what your lecturer meant by "every message should have its own tag".

Tag	Direction	When it's sent	What's inside
REPORT	Node → base station	A node is almost full **and** all its neighbours are full too	The congestion alert, sent as one message: timestamp, node ID and (x, y), each port's state, each neighbour's free ports, how many messages and bytes were exchanged with neighbours
REPLY	Base station → node	Straight after the base station reads a REPORT	Either the list of nearest nodes that have room, or "no nodes available nearby"
TERM (terminate)	Base station → every node	Once, when the simulation time is up	"Stop now", plus the time it was sent (used to measure delay)
DONE	Node → base	Once, after the ↓	"I've stopped", plus that node's statistics

- history[16] ring buffer of past availability

what does this mean?

Explaining how the ring buffer tracks node availability history.

It's a second small shared array that remembers the node's recent past. `history[16]` holds the number of free ports for the **last 16 cycles**. Each cycle, the comm thread writes the latest count into it, in the same snapshot that reads `port_state[]` (`snapshot_availability()` in `wsn_ver3.c`).

"Ring buffer" describes how it fills up. Once all 16 slots are used, the next value overwrites the oldest one and the writing goes back round to the start, which is why it's called a "ring". So it always holds exactly the last 16 values, its memory never grows, and you never have to shift or free anything. In code, that's a single line: `history[history_count % 16] = free_ports;`.

11:46 AM

</> task2_base.c

why should we use comm split

The core reason: you need a communicator whose members are exactly the grid nodes, and nothing else.

1. `MPI_Cart_create` **consumes every rank in the input communicator.** If you passed `MPI_COMM_WORLD` with `size = nx*ny + 1`, the dims wouldn't match the group size — Cart_create either errors or leaves the extra rank out as `MPI_COMM_NULL` in a way you can't control cleanly. Splitting first gives a group of exactly `nx*ny` ranks, so the grid dims are guaranteed to fit. Using `MPI_UNDEFINED` as the colour for `BASE_RANK` is exactly the idiom for "this rank joins no sub-communicator" — it gets `MPI_COMM_NULL` back and simply doesn't participate.
2. **Collectives would deadlock otherwise.** Node-side code does barriers/allreduce/neighbour exchange over `cart`. A collective on `MPI_COMM_WORLD` requires *all* ranks including the base station to call it. The base station is in its own event loop doing `MPI_Recv` / `Iprobe` on reports — it will never reach your barrier. Two disjoint communicators means the nodes can synchronise among themselves while the base station independently services messages.

wait since we find out the pre-posted Irecv is faster, do u think we should change our approach to that or no need?

Weighing whether deviating from the lecturer's instructions is justified.

I'd keep `Iprobe` → `Recv` as the default and present the pre-posted `Irecv` as your measured best option. Three reasons:

1. **Your lecturer explicitly asked for `Iprobe`.** In his notes: "Use Iprobe to check whether there is any incoming or not… only you call receive." Switching the default to a different pattern risks a marker seeing it as ignoring the instruction, even though it's faster.
2. **Your Task 3 data matches the default.** All 94 grid and line runs (graphs 1–5) used `Iprobe`. Changing the default now means either re-running everything on CAAS (about 13 more jobs, roughly 2+ hours with the queue) or presenting results from a different setup than the code you submit.
3. **The rubric doesn't require using the fastest version.** At HD it asks that multiple versions, blocking and non-blocking, are "implemented, explored, examined, and experimented to determine the best approach". You've done exactly that: three listeners, 36 runs, a clear winner. What earns the marks is the experiment and the conclusion.

Screenshots of Job running on CAAS

```
JOBID               JOBNAME    USERNAME      STATE          CORE   WCLIMIT  QUEUETIME
=============================================================================================
196556              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196557              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196558              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196559              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196560              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196561              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196562              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196563              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196564              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196565              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196566              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
196567              wsn_sweep  nnaz0017      Dependency     32     0:10:00  Wed Oct 21 13:36:04
```

```
[nnaz0017@student-caas-headnode task1]$ sbatch --export=ALL,SUITE="grid line",DURATION=60 wsn_sweep.job
Submitted batch job 195378
[nnaz0017@student-caas-headnode task1]$ sbatch --export=ALL,SUITE=grid,VARIANT=1,DURATION=60 wsn_sweep.job
Submitted batch job 195379
```